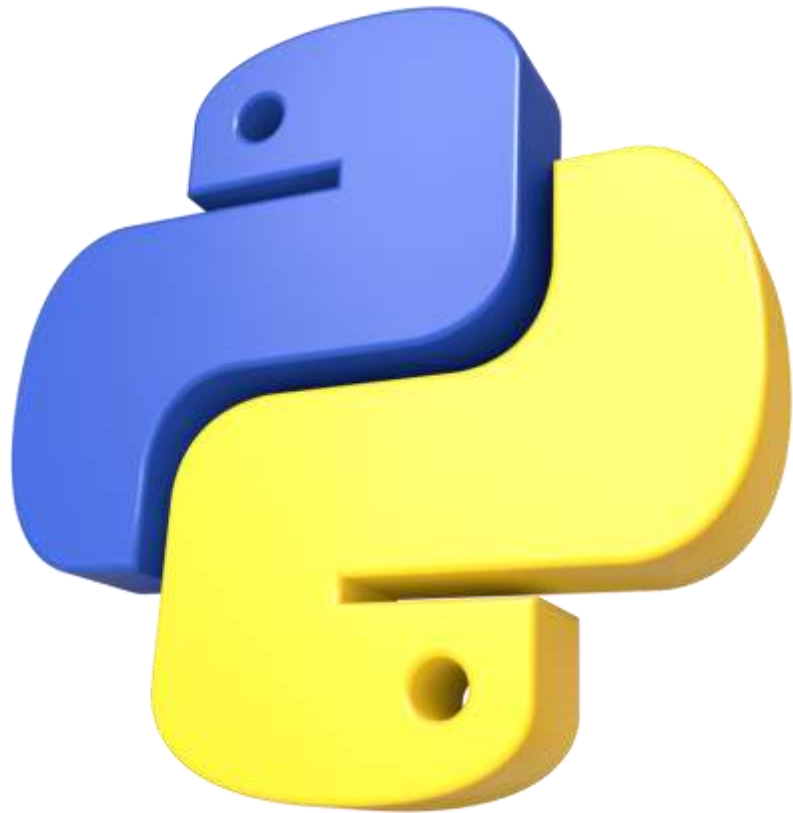




Aula 2

Prof. Gabriel Rodrigues e Wilson
Hissamu

Algoritmo e Programação

Temas desta aula

- Tipos de dados e variáveis
- Operações matemáticas básicas
- Concatenação de strings
- Saída de dados

INTRODUÇÃO

- Python é uma das linguagens de programação mais atualmente, amplamente aclamada por sua simplicidade, legibilidade e ecossistema de bibliotecas e estruturas para **ciência de dados, inteligência artificial e aprendizado de máquina.**
- Além disso, o Python oferece suporte a muitos tipos de dados, o que o torna uma ótima ferramenta para resolver problemas complexos e manipular dados. Ele ajuda os desenvolvedores a criar aplicativos úteis para o mundo real.

Tipos de dados Python

- Os tipos de dados em Python se enquadram em várias categorias, cada uma delas adaptada para lidar com tipos específicos de dados e operações.
- Entender essas categorias é um exercício teórico e uma habilidade prática que ajudará você a escrever um código eficiente e sem bugs. A seleção do tipo de dados correto pode afetar significativamente o **uso da memória , a velocidade de computação e a clareza do código.**

Tipos de dados Python

- O Python oferece uma variedade de tipos de dados incorporados.
- A seguir veremos uma tabela com a categorização dos tipos de dados.

Tipos de dados Python

Categoria	Tipos de dados
Numérico	int, float, complexo
Sequência	str, lista, tupla
Mapeamento	ditar
Conjunto	set, frozenset
Booleano	bool
NoneType	NoneType

Digitação dinâmica Python

- Python é uma **linguagem de programação tipada dinamicamente**, o que significa que você não precisa declarar explicitamente o tipo de uma variável no momento em que a define.
- Em vez disso, o interpretador Python determina e atribui um tipo dinamicamente com base no valor fornecido à variável.

Digitação dinâmica Python

- Para você entender como o Python é diferente, vamos compará-lo com uma linguagem estaticamente tipada como **Java**.
- Em Java, você deve declarar explicitamente o tipo de cada variável, conforme mostrado abaixo:

```
        int x = 10;           //  
Integer  
        double y = 10.5;    //  
Float  
        String z = "Hello";  //  
String
```


Digitação dinâmica Python

- O Python permite que você atribua valores diretamente sem especificar o tipo, o que o torna mais flexível e conciso:

```
x = 10 # Integer
```

```
y = 10.5 # Float
```

```
z = "Hello" # String
```

Digitação dinâmica Python

- No entanto, essa flexibilidade exige que você esteja atento a alterações de tipo não intencionais durante o tempo de execução, o que pode levar a erros.
- A digitação dinâmica é um dos motivos pelos quais o Python é tão popular entre os iniciantes, especialmente porque oferece um equilíbrio entre simplicidade e potência.

Tipos de dados numéricos int Python

- Números inteiros, positivos ou negativos, sem um componente fracionário.
- Esses são valores numéricos fundamentais que representam contagens ou medições discretas.
- Ao contrário dos números de ponto flutuante, os números inteiros são exatos e não envolvem aproximações, o que os torna ideais para cenários em que a precisão é importante, como indexação ou contagem de itens.

Tipos de dados numéricos float Python

- Números com pontos decimais ou em forma exponencial.
- Esses valores são comumente usados em cenários de precisão fracionária, como cálculos financeiros, medições científicas e programação de gráficos.
- Os números de ponto flutuante também podem representar valores extremamente grandes ou pequenos usando a notação exponencial, o que os torna versáteis para várias aplicações numéricas.

EX: `unifg = 2026.1`

Tipos de dados de sequência Python

- Além dos números, o Python também fornece tipos de dados de sequência que armazenam coleções ordenadas de elementos.
- **Strings (str) :**
 - Sequências ordenadas de caracteres entre aspas simples ou duplas. As cadeias de caracteres são um dos tipos de dados mais versáteis do Python, permitindo que você armazene e manipule dados baseados em texto.
 - Eles são comumente usados em tarefas como lidar com a entrada do usuário, criar mensagens ou até mesmo

Operações matemáticas Python

- As operações matemáticas básicas no Python são realizadas com operadores aritméticos nativos: adição (+), subtração (-), multiplicação (*), divisão (/), potência (**), divisão inteira (//) e módulo/resto (%).
- A ordem de precedência segue a matemática (parênteses, potências, multiplicações/divisões, adições/subtrações).

Operações matemáticas Python

Adição (+)

```
print(10 + 5) # 15
```

Subtração (-)

```
print(10 - 2) # 8
```

Multiplicação ()*

```
print(4 * 3) # 12
```

Divisão (/) - Retorna float

```
print(10 / 2) # 5.0
```

*# Potenciação (**)*

```
print(2 ** 3) # 8 (2 elevado a 3)
```

Divisão Inteira (//) - Descarta casas decimais

```
print(10 // 3) # 3
```

Módulo (%) - Resto da divisão

```
print(10 % 3) # 1
```

Exemplo com ordem de precedência

```
print((2 + 3) * 4**2) # (5 * 16) = 80
```

Operações matemáticas Python

Pontos importantes:

- **Divisão:** A barra simples `/` sempre retorna um número decimal (float), enquanto `//` retorna o inteiro.
- **Raízes:** Podem ser calculadas elevando o número a uma potência fracionária, por exemplo, `25 ** 0.5` para raiz quadrada.
- **Precedência:** Use parênteses `()` para garantir que uma operação seja realizada primeiro.

Concatenação de strings Python

- Concatenar é pegar duas strings e juntar elas pra formar uma nova string. Em Python, tem várias maneiras de juntar ou combinar strings. A nova string criada é chamada de objeto string.

Concatenação de strings Python

- **Usando o operador +**
 - Pra juntar duas strings em um único objeto, você pode usar o operador de concatenação (+). Esse método é simples e muito usado.
 - No exemplo a seguir, duas strings originais `my_string1` e `my_string2` são concatenadas. Para juntar as duas, você usa um operando de mais para “somar” as duas strings, colocando o espaço e gerando a saída.

Concatenação de strings Python

- Usando o operador +

```
my_string1 = "UNIFG"  
my_string2 = "Guanambi"  
combined_string = my_string1 + " " + my_string2  
print(combined_string)
```

```
# Expected output: UNIFG Guanambi
```